

```

import pandas as pd
import sqlite3

# Create an in-memory SQLite database connection
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

print("Setup complete. Environment ready.")

# -----
# UNNORMALIZED DATA (ONF)
# -----
data_Onf = {
    "Student_ID": [101, 101, 102, 103, 104],
    "Student_Name": ["Alice", "Alice", "Bob", "Charlie", "David"],
    "Course_Code": ["CS101", "MATH201", "CS101", "CS102", "MATH201"],
    "Course_Name": ["Intro to CS", "Calculus I", "Intro to CS", "Data Structures", "Calculus I"],
    "Instructor_ID": ["INS_01", "INS_02", "INS_01", "INS_03", "INS_02"],
    "Instructor_Name": ["Dr. Smith", "Dr. Jones", "Dr. Smith", "Dr. Alan", "Dr. Jones"],
    "Instructor_Office": ["Room 401", "Room 502", "Room 401", "Room 401", "Room 502"],
    "Textbooks_Required": [
        "Python Basics, Intro to CLI",
        "Calculus Vol 1",
        "Python Basics, Intro to CLI",
        "Algorithms Vol 1",
        "Calculus Vol 1"
    ],
    "Grade": ["A", "B", "A", "B+", "A-"]
}

df_Onf = pd.DataFrame(data_Onf)

print("\n--- Unnormalized Data (ONF) ---")
display(df_Onf)

```

Setup complete. Environment ready.

--- Unnormalized Data (ONF) ---

	Student_ID	Student_Name	Course_Code	Course_Name	Instructor_ID	Instructor_Name	Instructor_Office	Textbooks_Required
0	101	Alice	CS101	Intro to CS	INS_01	Dr. Smith	Room 401	Python Basics, Intro to CLI
1	101	Alice	MATH201	Calculus I	INS_02	Dr. Jones	Room 502	Calculus Vol 1
2	102	Bob	CS101	Intro to CS	INS_01	Dr. Smith	Room 401	Python Basics, Intro to CLI
3	103	Charlie	CS102	Data	INS_03	Dr. Alan	Room 401	Algorithms Vol 1

```

# Flattering the textbooks_required column to ensure atomocity

df_1nf = df_Onf.assign(Textbooks_Required = df_Onf['Textbooks_Required'].str.split(',')).explode('Textbooks_Required')

#Save_To_Sql
df_1nf.to_sql("Table_1NF", conn, index=False, if_exists='replace')
print("----1NF Data (Atomic values enforced)----")
print(f"Row count increased from {len(df_Onf)} to {len(df_1nf)} due to flattening")
df_1nf

```

----1NF Data (Atomic values enforced)----

Row count increased from 5 to 7 due to flattening

	Student_ID	Student_Name	Course_Code	Course_Name	Instructor_ID	Instructor_Name	Instructor_Office	Textbooks_Required
0	101	Alice	CS101	Intro to CS	INS_01	Dr. Smith	Room 401	Python Basics
0	101	Alice	CS101	Intro to CS	INS_01	Dr. Smith	Room 401	Intro to CLI
1	101	Alice	MATH201	Calculus I	INS_02	Dr. Jones	Room 502	Calculus Vol 1
2	102	Bob	CS101	Intro to CS	INS_01	Dr. Smith	Room 401	Python Basics
2	102	Bob	CS101	Intro to CS	INS_01	Dr. Smith	Room 401	Intro to CLI
3	103	Charlie	CS102	Data Structures	INS_03	Dr. Alan	Room 401	Algorithms Vol 1
4	104	David	MATH201	Calculus I	INS_02	Dr. Jones	Room 502	Calculus Vol 1

Student Activity

RUN THIS CELL TO SETUP THE CHALLENGE DATA

```
import pandas as pd
import sqlite3
```

```
conn_challenge = sqlite3.connect(':memory:')
```

```
challenge_data = {
    "Visit_ID": [5001, 5001, 5002, 5003],
    "Student_ID": [101, 101, 102, 104],
    "Student_Name": ["Alice", "Alice", "Bob", "David"],
    "Doctor_ID": ["DOC_XYZ", "DOC_XYZ", "DOC_ABC", "DOC_XYZ"],
    "Doctor_Name": ["Dr. Evans", "Dr. Evans", "Dr. Green", "Dr. Evans"],
    "Doctor_Clinic": ["General Medicine", "General Medicine", "Sports Med", "General Medicine"],
    "Prescriptions": ["Amoxicillin, Ibuprofen", "Amoxicillin, Ibuprofen", "Bandages", "Vitamin D"]
}
```

```
df_challenge = pd.DataFrame(challenge_data)
```

```
df_challenge.to_sql('Patient_Visits_ONF', conn_challenge, index=False, if_exists='replace')
```

```
print("--- Challenge Dataset (ONF) ---")
```

```
df_challenge
```

--- Challenge Dataset (ONF) ---

	Visit_ID	Student_ID	Student_Name	Doctor_ID	Doctor_Name	Doctor_Clinic	Prescriptions
0	5001	101	Alice	DOC_XYZ	Dr. Evans	General Medicine	Amoxicillin, Ibuprofen
1	5001	101	Alice	DOC_XYZ	Dr. Evans	General Medicine	Amoxicillin, Ibuprofen
2	5002	102	Bob	DOC_ABC	Dr. Green	Sports Med	Bandages
3	5003	104	David	DOC_XYZ	Dr. Evans	General Medicine	Vitamin D

Task-1 : Identify the Flaws

- Why does this table violate 1NF ? Which column is the culprit? -- *This table violates 1NF as Prescriptions column has a list of elements*
- Assume Visit_ID is the primary key for a flat version of this table. Why do Student_Name and Doctor_Clinic violate 2NF and 3NF?
 - 2NF and 3NF works on the principle that non-prime columns should be dependent on primary key(2NF) and should not be dependent on other non-prime keys(3NF). But Student_Name is depend on Student_ID and Doctor_Clinic is dependent on Doctor_ID/Doctor_Name that are non-prime violating 3NF. Neither of the columns are dependent on Visit_ID that is primary violating 2NF.*
- If the health center implements a rule where knowing the Doctor_ID immediatly tells you the Doctor_Clinic, but Doctor_ID is not a primary key of the registration table, which normal form is violated. -- *3NF is violated in this case, as it rules that any column should not be dependent on non-prime keys.*

Task-2 Fix it with Code

#1NF

```
# Flattering the textbooks_required column to ensure atomocity

df_c_inf = df_challenge.assign(Prescriptions = df_challenge['Prescriptions'].str.split(',')).explode('Prescriptions')

#Save_To_Sql
df_c_inf.to_sql("Table_1NF", conn, index=False, if_exists='replace')
print("----1NF Data (Atomic values enforced)----")
print(f"Row count increased from {len(df_challenge)} to {len(df_c_inf)} due to flattening")
df_c_inf
```

----1NF Data (Atomic values enforced)----

Row count increased from 4 to 6 due to flattening

	Visit_ID	Student_ID	Student_Name	Doctor_ID	Doctor_Name	Doctor_Clinic	Prescriptions
0	5001	101	Alice	DOC_XYZ	Dr. Evans	General Medicine	Amoxicillin
0	5001	101	Alice	DOC_XYZ	Dr. Evans	General Medicine	Ibuprofen
1	5001	101	Alice	DOC_XYZ	Dr. Evans	General Medicine	Amoxicillin
1	5001	101	Alice	DOC_XYZ	Dr. Evans	General Medicine	Ibuprofen
2	5002	102	Bob	DOC_ABC	Dr. Green	Sports Med	Bandages
3	5003	104	David	DOC_XYZ	Dr. Evans	General Medicine	Vitamin D